



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 11

1. ENUNCIADO

Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade. Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas `schedule` e `collapse` no desempenho da execução paralela.

2. DISCUSSÃO

A equação de Navier-Stokes descreve o movimento de um fluido e, na sua forma completa, leva em conta diversos fatores, como a pressão, a viscosidade, o termo de convecção e as forças externas. No entanto, nesta tarefa, foi solicitado considerar apenas os efeitos da viscosidade, desconsiderando a pressão, as forças externas e também o transporte por convecção. Dessa maneira, quando todos esses termos são retirados, a equação se reduz à equação da difusão (também conhecida como equação do calor):

$$\partial u / \partial t = \nu \nabla^2 u$$

Em outras palavras, a viscosidade funciona como um mecanismo que "espalha" a velocidade pelas regiões vizinhas, suavizando as diferenças até que o campo fique homogêneo. É exatamente esse comportamento que será observado: uma perturbação localizada deve se espalhar suavemente ao longo do tempo, ao invés de aparecer um pico abrupto ou de surgir instabilidades.

Vale destacar que a velocidade do fluido, a rigor, é um vetor (componentes em x e em y). Contudo, como estamos considerando apenas a viscosidade, cada componente obedece à mesma equação de difusão de forma independente. Dessa forma, para manter a implementação simples, foi simulado um único campo escalar de velocidade, que já é representativo do fenômeno.

2.1. Discretização por diferenças finitas

Como o computador não consegue trabalhar com o espaço contínuo, é preciso "quebrar" esse espaço em uma grade de pontos, com um espaçamento dx entre eles. Assim, ao invés de uma função contínua, tem-se uma matriz $N \times N$ de valores de velocidade.

Para aproximar a segunda derivada (o Laplaciano $\nabla^2 u$), foi utilizado o esquema de diferenças finitas centrais, que considera os quatro vizinhos de cada ponto (cima, baixo, esquerda e direita):

$$\nabla^2 u \approx u[i+1][j] + u[i-1][j] + u[i][j+1] + u[i][j-1] - 4 \cdot u[i][j]$$

Já o avanço no tempo foi feito pelo método de Euler explícito, no qual o valor novo de cada ponto depende apenas dos valores antigos:

$$u_novo[i][j] = u[i][j] + \alpha \cdot (u[i+1][j] + u[i-1][j] + u[i][j+1] + u[i][j-1] - 4 \cdot u[i][j])$$

em que $\alpha = v \cdot dt/dx^2$ é chamado de número de difusão.

2.2. Condição de estabilidade

Por mais que o método explícito seja simples de programar, ele só funciona corretamente se o passo de tempo não for grande demais. Caso contrário, os valores começam a oscilar e a explodir, gerando uma simulação instável. Em duas dimensões, a condição de estabilidade exige que:

$$\alpha = v \cdot dt/dx^2 \leq 0,25$$

Dessa maneira, nesta tarefa foram escolhidos $v = 0,2$, $dx = 1,0$ e $dt = 1,0$, resultando em $\alpha = 0,2$, valor que está abaixo do limite e, portanto, garante uma simulação estável.

Além disso, foi necessário usar duas grades separadas (u e u_novo), porque, se a atualização fosse feita na mesma matriz, alguns vizinhos já estariam atualizados e outros não, misturando valores antigos e novos no mesmo passo. Por isso, ao final de cada passo, os ponteiros das duas grades são apenas trocados (técnica de double buffer).

2.3. Paralelização com OpenMP

Após validar o código, ele foi paralelizado com a diretiva `#pragma omp parallel for`, aplicada ao laço que percorre os pontos da grade. Como cada ponto novo é calculado de forma independente a partir dos valores antigos, não há dependência entre as iterações, o que torna esse laço um ótimo candidato para paralelização.

Em relação às cláusulas exploradas, tem-se:

- `schedule(static)`: divide as iterações em blocos iguais e distribui uma única vez no início. Como cada ponto da grade custa o mesmo tempo de cálculo (carga uniforme), essa costuma ser a opção mais eficiente, já que tem baixo custo de gerenciamento.
- `schedule(dynamic)`: cada thread pega um pedaço pequeno e, quando termina, pega outro. É útil quando o trabalho é desigual, mas, neste caso, como a carga é uniforme, tende a adicionar um custo de gerenciamento desnecessário.
- `schedule(guided)`: semelhante ao `dynamic`, porém começa com pedaços grandes e vai diminuindo o tamanho deles. Fica num meio-termo entre os dois anteriores.
- `collapse(2)`: junta os dois laços aninhados (linhas e colunas) em um único espaço de iteração maior. Isso ajuda quando o laço externo tem poucas iterações para distribuir entre muitas threads. Todavia, neste programa o laço externo já possui quase N iterações, número mais que suficiente, então o `collapse` pode acabar adicionando custo sem trazer ganho real.

2.4. Etapas para resolução da tarefa

- 1º) Montar a grade $N \times N$ e os parâmetros físicos (v , dx , dt);
- 2º) Implementar o passo no tempo com diferenças finitas (versão sequencial);
- 3º) Validar a física: testar fluido parado (deve continuar parado) e testar a perturbação (deve se difundir suave);
- 4º) Paralelizar o laço principal com `#pragma omp parallel for`;
- 5º) Criar versões com `schedule(static)`, `schedule(dynamic)` e `schedule(guided)`;
- 6º) Criar uma versão com `collapse(2)`;
- 7º) Medir o tempo de cada versão e calcular o speedup;
- 8º) Comparar os resultados e analisar o impacto de cada cláusula.

2.5. ETAPAS PARA RESOLUÇÃO DA TAREFA

2.6. COMPILAR

```
gcc -fopenmp tarefa11.c -o tarefa11  
$env:OMP_NUM_THREADS=4  
.\tarefa11.exe
```

2.7. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.8. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

A simulação confirmou que, ao manter apenas o termo de viscosidade, a equação de Navier-Stokes se comporta como uma equação de difusão: o fluido em repouso permaneceu estável e a perturbação se espalhou de forma suave ao longo do tempo, validando tanto a discretização por diferenças finitas quanto a escolha do passo de tempo dentro do limite de estabilidade.

Na parte da paralelização, ficou claro que a melhor estratégia depende do tipo de carga do problema. Como esse programa tem carga uniforme, o `schedule(static)` se mostrou a escolha mais adequada, enquanto o `dynamic` e o `guided` adicionaram custo sem benefício. O `collapse(2)`, por sua vez, só compensaria caso o laço externo tivesse poucas iterações. Por fim, os resultados reforçam que, antes de escolher uma cláusula de paralelização, é essencial entender como o trabalho está distribuído no problema, já que uma cláusula mal escolhida pode até piorar o desempenho.

4. APÊNDICE

4.1. RESOLUÇÃO

```
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>  
#include <omp.h>
```

```

// Tamanho da grade NxN -> o espaço fica quebrado em pontinhos (diferenças finitas)
#define N 1000

// Quantos passos no tempo a simulação vai dar (quantas vezes o fluido anda no tempo)
#define STEPS 600

// Parâmetros físicos da simulação
#define NU 0.2 // viscosidade do fluido
#define DX 1.0 // distância entre dois pontos vizinhos da grade
#define DT 1.0 // tamanho do passo de tempo

// ALFA é o número de difusão:  $\nu * dt / dx^2$ 
// pra simulação ser estável em 2D, ALFA tem que ser  $\leq 0.25$ , então 0.2 tá ok
#define ALFA (NU * DT / (DX * DX))

// Função pra reservar a grade na memória (vetor 1D de tamanho N*N usado como se fosse 2D)
double *aloca_grade() {
    double *g = (double *) malloc((N * N) * sizeof(double));
    if (g == NULL) {
        printf("Erro: Faltou memória!\n");
        exit(1);
    }
    return g;
}

// Deixa o fluido todo parado (velocidade = 0 em todo lugar)
void inicializa_repouso(double *u) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            u[i * N + j] = 0.0;
        }
    }
}

// Deixa o fluido parado mas coloca uma perturbação (um soco de velocidade) no meio
void inicializa_perturbacao(double *u) {
    inicializa_repouso(u); // primeiro zera tudo
    // coloca um valor alto no centro da grade pra ver se ele se espalha suave
    int meio = N / 2;
    u[meio * N + meio] = 100.0;
}

```

```

// Um passo no tempo sem paralelizar (versão sequencial)
// usa o vizinho de cima, de baixo, da esquerda e da direita pra calcular o
novo valor
void passo_sequencial(double *u, double *u_novo) {
    for (int i = 1; i < N - 1; i++) {
        for (int j = 1; j < N - 1; j++) {
            u_novo[i * N + j] = u[i * N + j] + ALFA * (u[(i + 1) * N + j] +
u[(i - 1) * N + j] + u[i * N + (j + 1)] + u[i * N + (j - 1)] - 4.0 * u[i *
N + j]);
        }
    }
}

// VALIDAÇÃO DA FÍSICA (tudo sequencial aqui)
void validacao() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();

    // TESTE 1: fluido parado tem que continuar parado
    inicializa_repouso(u);
    for (int t = 0; t < STEPS; t++) {
        passo_sequencial(u, u_novo);
        double *tmp = u; u = u_novo; u_novo = tmp; // troca os ponteiros
(double buffer)
    }
    // procura o maior valor pra ver se continuou tudo zero
    double maior = 0.0;
    for (int i = 0; i < N * N; i++) {
        if (fabs(u[i]) > maior) maior = fabs(u[i]);
    }
    printf("VALIDACAO\n");
    printf("Teste fluido parado -> maior velocidade no fim: %f
(esperado)\n\n", maior);

    // TESTE 2: coloca uma perturbação no meio e vê se ela se difunde suave
    inicializa_perturbacao(u);
    int meio = N / 2;
    printf("Teste perturbacao (valor no centro ao longo do tempo):\n");
    for (int t = 0; t <= STEPS; t++) {
        // imprime de vez em quando pra acompanhar o espalhamento
        if (t % 150 == 0) {
            printf(" passo %4d -> centro: %8.4f | vizinho a 5
casas: %8.4f\n", t, u[meio * N + meio], u[meio * N + (meio + 5)]);
        }
        passo_sequencial(u, u_novo);
        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    printf("\n");
}

```

```

    free(u);
    free(u_novo);
}

// VERSÕES PRA MEDIR TEMPO

// Sequencial (sem OpenMP) -> serve de referência pro speedup
double simular_sequencial() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();
    inicializa_perturbacao(u);

    double inicio = omp_get_wtime();
    for (int t = 0; t < STEPS; t++) {
        passo_sequencial(u, u_novo);
        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    double fim = omp_get_wtime();

    free(u);
    free(u_novo);
    return fim - inicio;
}

// Paralela básica: só joga o #pragma omp parallel for no laço de fora
double simular_parallel_for() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();
    inicializa_perturbacao(u);

    double inicio = omp_get_wtime();
    for (int t = 0; t < STEPS; t++) {
        #pragma omp parallel for
        for (int i = 1; i < N - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                u_novo[i * N + j] = u[i * N + j] + ALFA * (u[(i + 1) * N +
j] + u[(i - 1) * N + j] + u[i * N + (j + 1)] + u[i * N + (j - 1)] - 4.0 *
u[i * N + j]);
            }
        }
        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    double fim = omp_get_wtime();

    free(u);
    free(u_novo);
    return fim - inicio;
}

```

```

}

// schedule(static): divide as linhas em blocos iguais, uma vez só, no
começo
double simular_static() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();
    inicializa_perturbacao(u);

    double inicio = omp_get_wtime();
    for (int t = 0; t < STEPS; t++) {
        #pragma omp parallel for schedule(static)
        for (int i = 1; i < N - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                u_novo[i * N + j] = u[i * N + j] + ALFA * (u[(i + 1) * N +
j] + u[(i - 1) * N + j] + u[i * N + (j + 1)] + u[i * N + (j - 1)] - 4.0 *
u[i * N + j]);
            }
        }
        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    double fim = omp_get_wtime();

    free(u);
    free(u_novo);
    return fim - inicio;
}

// schedule(dynamic): cada thread pega um pedacinho, quando termina pega
outro
double simular_dynamic() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();
    inicializa_perturbacao(u);

    double inicio = omp_get_wtime();
    for (int t = 0; t < STEPS; t++) {
        #pragma omp parallel for schedule(dynamic)
        for (int i = 1; i < N - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                u_novo[i * N + j] = u[i * N + j] + ALFA * (u[(i + 1) * N +
j] + u[(i - 1) * N + j] + u[i * N + (j + 1)] + u[i * N + (j - 1)] - 4.0 * u[i
* N + j]);
            }
        }
        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    double fim = omp_get_wtime();
}

```

```

    free(u);
    free(u_novo);
    return fim - inicio;
}

// schedule(guided): começa com pedaços grandes e vai diminuindo
double simular_guided() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();
    inicializa_perturbacao(u);

    double inicio = omp_get_wtime();
    for (int t = 0; t < STEPS; t++) {
        #pragma omp parallel for schedule(guided)
        for (int i = 1; i < N - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                u_novo[i * N + j] = u[i * N + j] + ALFA * (
                    u[(i + 1) * N + j] + u[(i - 1) * N + j] + u[i * N + (j +
1)] + u[i * N + (j - 1)] - 4.0 * u[i * N + j]);
            }
        }
        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    double fim = omp_get_wtime();

    free(u);
    free(u_novo);
    return fim - inicio;
}

// collapse(2): junta os dois laços (i e j) num só, dando mais trabalho pra
dividir
double simular_collapse() {
    double *u = aloca_grade();
    double *u_novo = aloca_grade();
    inicializa_perturbacao(u);

    double inicio = omp_get_wtime();
    for (int t = 0; t < STEPS; t++) {
        #pragma omp parallel for schedule(static) collapse(2)
        for (int i = 1; i < N - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                u_novo[i * N + j] = u[i * N + j] + ALFA * (u[(i + 1) * N +
j] + u[(i - 1) * N + j] + u[i * N + (j + 1)] + u[i * N + (j - 1)] - 4.0 * u[i
* N + j]);
            }
        }
    }
}

```

```

        double *tmp = u; u = u_novo; u_novo = tmp;
    }
    double fim = omp_get_wtime();

    free(u);
    free(u_novo);
    return fim - inicio;
}

int main() {
    printf("Grade: %dx%d | Passos no tempo: %d | Threads: %d |  

    ALFA: %.3f\n\n", N, N, STEPS, omp_get_max_threads(), ALFA);

    // PARTE 1: validar se a física tá certa
    validacao();

    // PARTE 2: medir tempo das versões e calcular speedup
    printf("DESEMPENHO\n");
    double t_seq = simular_sequencial();
    printf("Sequencial -> %.4f s\n", t_seq);

    double t_par = simular_parallel_for();
    printf("parallel for -> %.4f s | speedup: %.2fx\n", t_par, t_seq /
t_par);

    double t_static = simular_static();
    printf("schedule(static) -> %.4f s | speedup: %.2fx\n", t_static, t_seq
/ t_static);

    double t_dynamic = simular_dynamic();
    printf("schedule(dynamic) -> %.4f s | speedup: %.2fx\n", t_dynamic,
t_seq / t_dynamic);

    double t_guided = simular_guided();
    printf("schedule(guided) -> %.4f s | speedup: %.2fx\n", t_guided, t_seq
/ t_guided);

    double t_collapse = simular_collapse();
    printf("static + collapse(2) -> %.4f s | speedup: %.2fx\n", t_collapse,
t_seq / t_collapse);

    return 0;
}

```

4.2. OUTPUT

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11> $env:OMP_NUM_THREADS=4
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11> .\tarefa11.exe
Grade: 1000x1000 | Passos no tempo: 600 | Threads: 4 | ALFA: 0.200

VALIDACAO
Teste fluido parado -> maior velocidade no fim: 0.000000 (esperado)

Teste perturbacao (valor no centro ao longo do tempo):
passo 0 -> centro: 100.0000 | vizinho a 5 casas: 0.0000
passo 150 -> centro: 0.2646 | vizinho a 5 casas: 0.2151
passo 300 -> centro: 0.1325 | vizinho a 5 casas: 0.1194
passo 450 -> centro: 0.0883 | vizinho a 5 casas: 0.0824
passo 600 -> centro: 0.0663 | vizinho a 5 casas: 0.0629

DESEMPENHO
Sequencial -> 1.9190 s
parallel for -> 0.5580 s | speedup: 3.44x
schedule(static) -> 0.5690 s | speedup: 3.37x
schedule(dynamic) -> 0.5990 s | speedup: 3.20x
schedule(guided) -> 0.5780 s | speedup: 3.32x
static + collapse(2) -> 0.5880 s | speedup: 3.26x
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11> $env:OMP_NUM_THREADS=5
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11> .\tarefa11.exe
Grade: 1000x1000 | Passos no tempo: 600 | Threads: 5 | ALFA: 0.200

VALIDACAO
Teste fluido parado -> maior velocidade no fim: 0.000000 (esperado)

Teste perturbacao (valor no centro ao longo do tempo):
passo 0 -> centro: 100.0000 | vizinho a 5 casas: 0.0000
passo 150 -> centro: 0.2646 | vizinho a 5 casas: 0.2151
passo 300 -> centro: 0.1325 | vizinho a 5 casas: 0.1194
passo 450 -> centro: 0.0883 | vizinho a 5 casas: 0.0824
passo 600 -> centro: 0.0663 | vizinho a 5 casas: 0.0629

DESEMPENHO
Sequencial -> 1.9370 s
parallel for -> 0.4830 s | speedup: 4.01x
schedule(static) -> 0.4800 s | speedup: 4.04x
schedule(dynamic) -> 0.5110 s | speedup: 3.79x
schedule(guided) -> 0.4800 s | speedup: 4.04x
static + collapse(2) -> 0.4900 s | speedup: 3.95x
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11> $env:OMP_NUM_THREADS=8
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11> .\tarefa11.exe
Grade: 1000x1000 | Passos no tempo: 600 | Threads: 8 | ALFA: 0.200

VALIDACAO
Teste fluido parado -> maior velocidade no fim: 0.000000 (esperado)

Teste perturbacao (valor no centro ao longo do tempo):
passo 0 -> centro: 100.0000 | vizinho a 5 casas: 0.0000
passo 150 -> centro: 0.2646 | vizinho a 5 casas: 0.2151
passo 300 -> centro: 0.1325 | vizinho a 5 casas: 0.1194
passo 450 -> centro: 0.0883 | vizinho a 5 casas: 0.0824
passo 600 -> centro: 0.0663 | vizinho a 5 casas: 0.0629

DESEMPENHO
Sequencial -> 1.9200 s
parallel for -> 0.4570 s | speedup: 4.20x
schedule(static) -> 0.4330 s | speedup: 4.43x
schedule(dynamic) -> 0.4020 s | speedup: 4.78x
schedule(guided) -> 0.3730 s | speedup: 5.15x
static + collapse(2) -> 0.4530 s | speedup: 4.24x
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa11>
```

